`model/tpm.py` (Q&A Documentation)

Q: What does this file implement at a high level? **A:** This file implements TPM-style output: the model predicts probabilities for a binary tree over buckets, which can be decoded into an expected watch-time.

Q: Why does TPM return a vector instead of a scalar? **A:** TPM represents the label as a structured set of binary decisions (tree nodes). This provides many supervised signals per sample and enables decoding into an expected value and an uncertainty proxy.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
class TPM  # methods: __init__, build, init, forward
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import torch

from model.layers import  MultiLayerPerceptronTPM

class TPM(torch.nn.Module):

    def __init__(self, description, class_num, embed_dim, mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'se
        self.build(embed_dim, mlp_dims, dropout,class_num)

    def build(self, embed_dim, mlp_dims, dropout,class_num):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()
        self.ctn_linear_layer = torch.nn.ModuleDict()
        embed_output_dim = 0
        for name, (size, type) in self.features.items():
            if type == 'spr':
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
                embed_output_dim += embed_dim
            elif type == 'ctn':
                self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
            elif type == 'seq':
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
```

```
                embed_output_dim += embed_dim
            else:
                raise ValueError('unkown feature type: {}'.format(type))
        self.mlp = MultiLayerPerceptronTPM(embed_output_dim, mlp_dims, dropout, class_num)
        return

    def init(self):
        for param in self.parameters():
            torch.nn.init.uniform_(param, -0.01, 0.01)

    def forward(self, x_dict):
        linears = []
        embs = []
        for name, (_, type) in self.features.items():
            x = x_dict[name]
            if type == 'spr':
                embs.append(self.emb_layer[name](x).squeeze(1))
            elif type == 'ctn':
                linears.append(self.ctn_linear_layer[name](x))
            elif type == 'seq':
                seq_emb = self.emb_layer[name](x)
                seq_mask = torch.unsqueeze(x_dict["{}mask".format(name)], dim=2)
                embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
            else:
                raise ValueError('unkwon feature: {}'.format(name))
        emb = torch.cat(embs, dim=1)
```

Q: What is `TPM` and what responsibility does it have? **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What are the important methods on `TPM`? **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```
__init__
build
init
forward
```

Q: What does the implementation look like for `TPM`? **A:** Excerpt from the file:

```
class TPM(torch.nn.Module):

    def __init__(self, description, class_num, embed_dim, mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'seq', 'spr'])}
        self.build(embed_dim, mlp_dims, dropout,class_num)

    def build(self, embed_dim, mlp_dims, dropout,class_num):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()
        self.ctn_linear_layer = torch.nn.ModuleDict()
        embed_output_dim = 0
        for name, (size, type) in self.features.items():
            if type == 'spr':
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
                embed_output_dim += embed_dim
```

```
            elif type == 'ctn':
                self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
            elif type == 'seq':
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
                embed_output_dim += embed_dim
            else:
                raise ValueError('unkown feature type: {}'.format(type))
        self.mlp = MultiLayerPerceptronTPM(embed_output_dim, mlp_dims, dropout, class_num)
        return

    def init(self):
        for param in self.parameters():
            torch.nn.init.uniform_(param, -0.01, 0.01)

    def forward(self, x_dict):
        linears = []
        embs = []
        for name, (_, type) in self.features.items():
            x = x_dict[name]
            if type == 'spr':
                embs.append(self.emb_layer[name](x).squeeze(1))
            elif type == 'ctn':
                linears.append(self.ctn_linear_layer[name](x))
            elif type == 'seq':
                seq_emb = self.emb_layer[name](x)
                seq_mask = torch.unsqueeze(x_dict["{}mask".format(name)], dim=2)
                embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
            else:
                raise ValueError('unkwon feature: {}'.format(name))
        emb = torch.cat(embs, dim=1)
        res = self.mlp(emb)
        return torch.sigmoid(res)
```

Q: Why is `TPM` needed by training scripts? **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```
for features, label in dataloaders["train"]:
    out = model(features)
    loss = ...
    loss.backward(); optimizer.step()
```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```
import torch

from model.layers import  MultiLayerPerceptronTPM

class TPM(torch.nn.Module):

    def __init__(self, description, class_num, embed_dim, mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'seq', 'spr'])}
        self.build(embed_dim, mlp_dims, dropout,class_num)

    def build(self, embed_dim, mlp_dims, dropout,class_num):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()
        self.ctn_linear_layer = torch.nn.ModuleDict()
```

```
    embed_output_dim = 0
    for name, (size, type) in self.features.items():
        if type == 'spr':
            self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
            embed_output_dim += embed_dim
        elif type == 'ctn':
            self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
        elif type == 'seq':
            self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
            embed_output_dim += embed_dim
        else:
            raise ValueError('unkown feature type: {}'.format(type))
    self.mlp = MultiLayerPerceptronTPM(embed_output_dim, mlp_dims, dropout, class_num)
    return

def init(self):
    for param in self.parameters():
        torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
        else:
            raise ValueError('unkwon feature: {}'.format(name))
    emb = torch.cat(embs, dim=1)
    res = self.mlp(emb)
    return torch.sigmoid(res)
```